

Compte-rendu — SAE4D01 DevCloud / Datacenters

URTADO Pierre — Groupe 8 — BUT R&T, IUT Béziers

Fabric datacenter **leaf-spine** décliné en 4 technologies de routage, comparé par **benchmark chiffré** et supervisé en **temps réel (Grafana)**. Lab 100 % **containerlab + FRR**.

1. Infrastructure & vocabulaire

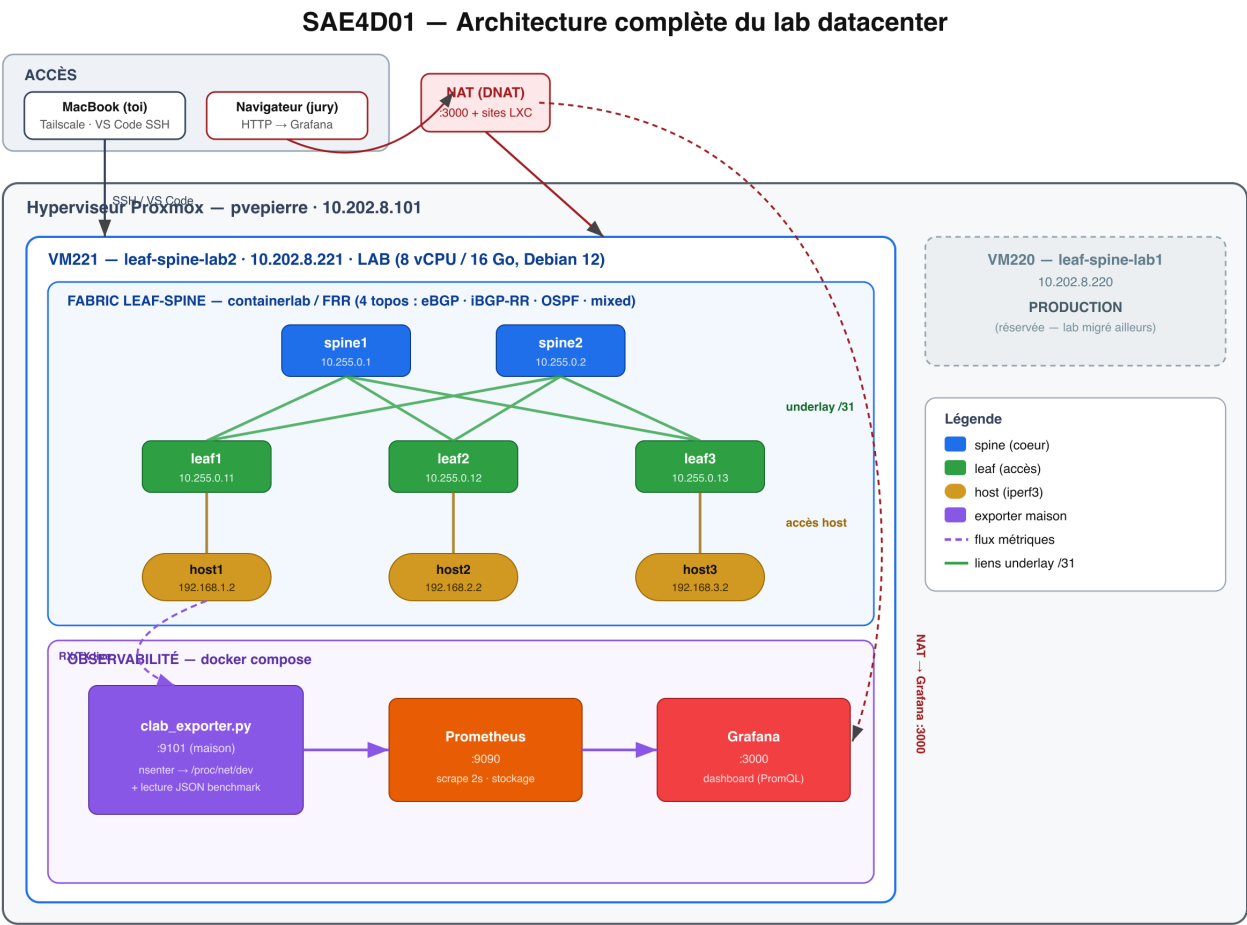


Figure 1: Architecture complète

Plateforme : Proxmox □ VM dédiée **VM221** (leaf-spine-lab2, 8 vCPU / 16 Go / 60 Go, Debian 12). **Outils :** **containerlab 0.76.1** (topologies déclaratives en YAML) + **FRR 10.6.1** (suite de routage open-source : bgpd, ospfd, zebra) + **Alpine** (hosts iperf3).

Architecture leaf-spine (Clos) : chaque leaf est relié à **chaque** spine ; les leaf ne se parlent jamais directement. Tout flux host □ host = **2 sauts** (latence prévisible), **tous les liens actifs** (ECMP, pas de Spanning-Tree).

Vocabulaire clé du datacenter (utilisé dans tout le CR) :

Terme	Définition
Underlay	le réseau physique/IP qui assure la connectivité de base (les liens /31 spine-leaf)

Terme	Définition
Overlay	un réseau logique monté <i>par-dessus</i> l'underlay (ex : sessions iBGP via loopbacks, VXLAN)
Ports	nommage des interfaces : leaf eth1→spine1, eth2→spine2, eth3→host ; spine eth1/2/3→leaf1/2/3
Loopback	interface virtuelle toujours active (lo0), identité stable du routeur (router-id, VTEP)

Plan d'adressage (commun aux 4 topos) : - Liens underlay **/31** (RFC 3021, 2 IP exactes) : spine1→leaf1/2/3 = 10.0.1.0 / .2 / .4, spine2→leaf1/2/3 = 10.0.1.6 / .8 / .10 - Loopbacks **/32** : spine1 10.255.0.1, spine2 10.255.0.2, leaf1-3 10.255.0.11-13 - Hosts : 192.168.1.2 / 2.2 / 3.2

2. Méthodologie

Pour chaque techno, démarche identique : **déployer** □ **vérifier l'état du protocole** □ **vérifier la connectivité** □ **mesurer le débit**. Déploiement reproductible (containerlab deploy), benchmark automatisé (benchmark.sh : convergence 35 s □ iperf3 TCP 30 s + UDP 30 s □ résultats JSON).

3. Lab 1 — eBGP (baseline)

- **Objectif** : fabric « BGP-only » (RFC 7938), modèle des grands datacenters.
- **Plan** : un AS par routeur (spine1=65081, spine2=65082, leaf1-3=65083-85). Une **session eBGP par lien physique** (underlay = plan unique, pas d'overlay).
- **Config clé** (leaf1) :

```
router bgp 65083
 neighbor 10.0.1.1 remote-as 65081    ! vers spine1 (eth1)
 neighbor 10.0.1.7 remote-as 65082    ! vers spine2 (eth2)
```
- **Vérification** : show bgp summary (sessions Established), show ip route bgp (préfixes appris), ping host1 □ host3.

```
eBGP - show bgp summary (spine1)

$ docker exec clab-topo-ebgp-spine1 vtysh -c "show bgp summary"

IPv4 Unicast Summary:
BGP router identifier 10.255.0.1, local AS number 65081 VRF default vrf-id 0
BGP table version 8
RIB entries 15, using 2280 bytes of memory
Peers 3, using 50 KiB of memory

Neighbor V AS MsgRcvd MsgSent TblVer InQ OutQ Up/Down State/PfxRcd PfxSnt Desc
10.0.1.1 4 65083 9 8 8 0 0 00:00:51 7 8 leaf1
10.0.1.3 4 65084 8 8 8 0 0 00:00:51 3 8 leaf2
10.0.1.5 4 65085 8 8 8 0 0 00:00:51 3 8 leaf3

Total number of neighbors 3
```

- **Conclusion** : eBGP propage les routes naturellement via l'AS-path (anti-boucle), pas de full-mesh requis, next-hop réécrit automatiquement. Simple et scalable.

topo-ebgp — eBGP (un AS par routeur)

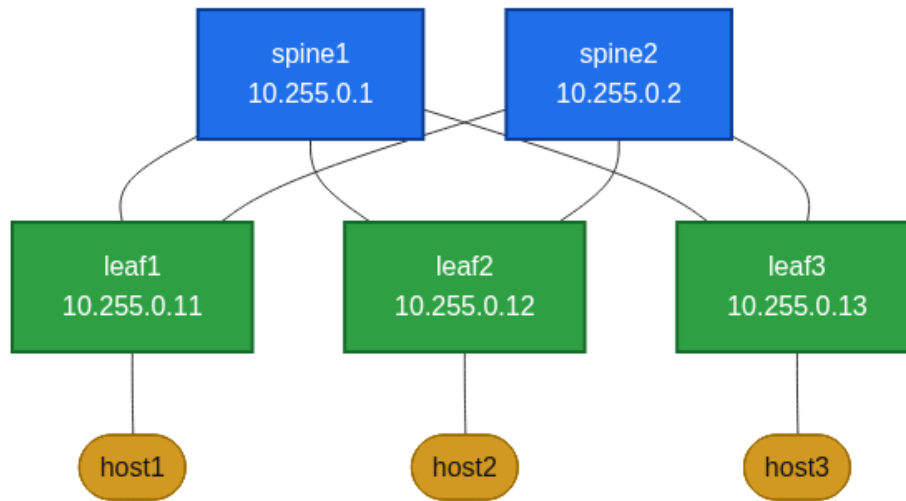


Figure 2: Topologie eBGP

4. Lab 2 — iBGP + Route Reflector

topo-ibgp-rr — iBGP + Route Reflector (AS 65080)

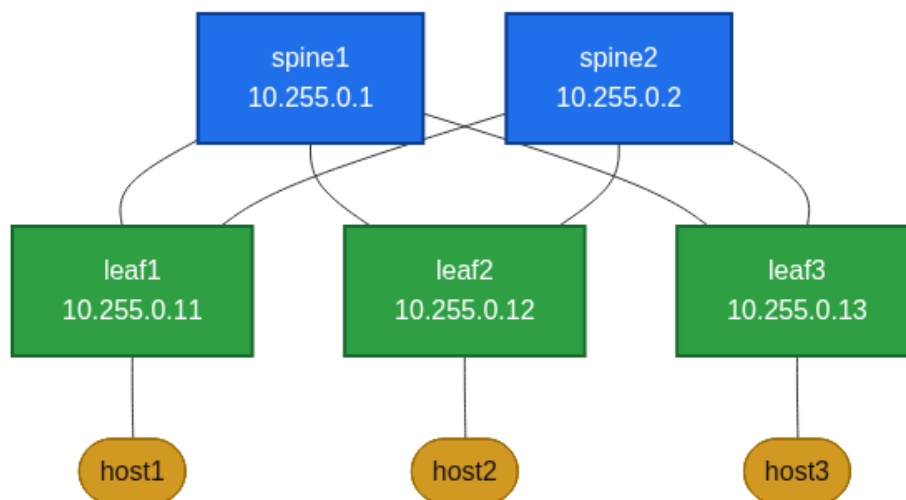


Figure 3: Topologie iBGP-RR

- **Objectif** : rester dans **un seul AS** sans subir le full-mesh iBGP ($N \times (N-1) / 2$ sessions).
- **Plan** : **AS 65080** unique. Les **spines = Route Reflectors**, les leafs = clients. Sessions iBGP montées sur les **IP de lien** (10.0.1.x) □ underlay, plan unique.
- **Config clé** (spine1, RR) :


```

router bgp 65080
 neighbor 10.0.1.1 route-reflector-client    ! reflète vers leaf1
 neighbor 10.0.1.1 next-hop-self force      ! cf. bug 1
            
```
- **Vérification** : show bgp summary (les leafs apprennent les routes des autres via le RR), ECMP sur les 2

spines.

- **Conclusion** : le RR évite le full-mesh ☐ scalable. Mais bug du next-hop (voir §8.1).

5. Lab 3 — OSPF

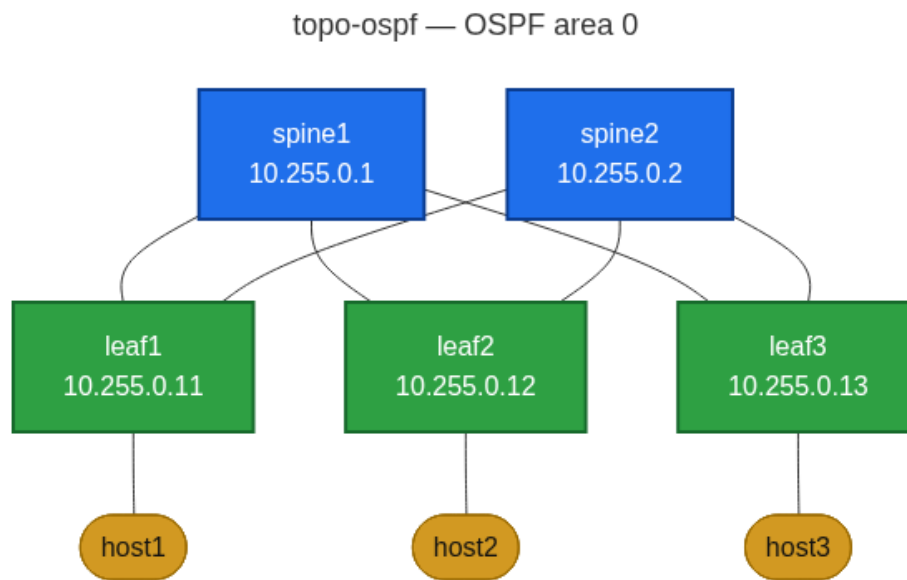


Figure 4: Topologie OSPF

- **Objectif** : IGP link-state pur (Dijkstra/SPF).
- **Plan** : **OSPF area 0** sur tous les liens underlay /31. Plan unique.
- **Config clé** (chaque interface underlay) :

```
interface eth1
  ip ospf network point-to-point ! cf. bug 2
```

- **Vérification** : `show ip ospf neighbor` (état Full), table de routage, ping.

```
OSPF - show ip ospf neighbor (spine1)

$ docker exec clab-topo-ospf-spine1 vtysh -c "show ip ospf neighbor"

Neighbor ID Pri State Up Time Dead Time Address Interface RXmtL RqstL DBsml
10.255.0.11 1 Full/- 25.754s 39.318s 10.0.1.1 eth1:10.0.1.0 2 0 0
10.255.0.12 1 Full/- 30.756s 39.245s 10.0.1.3 eth2:10.0.1.2 2 0 0
10.255.0.13 1 Full/- 25.754s 39.991s 10.0.1.5 eth3:10.0.1.4 2 0 0
```

- **Conclu-**

sion : convergence rapide, simple sur petit fabric. Flooding des LSA ☐ moins scalable que BGP à très grande échelle.

6. Lab 4 — Mixed (underlay OSPF + overlay iBGP) ☐

- **Objectif** : le **modèle réel des datacenters modernes** — séparer underlay et overlay.
- **Plan** :
 - **UNDERLAY** = **OSPF area 0** sur les liens /31 ☐ rend les **loopbacks** joignables entre eux.
 - **OVERLAY** = **iBGP AS 65080** (spines = RR), sessions montées **via les loopbacks** (update-source lo0), pas sur les IP de lien.

topo-mixed — OSPF underlay + iBGP-RR overlay

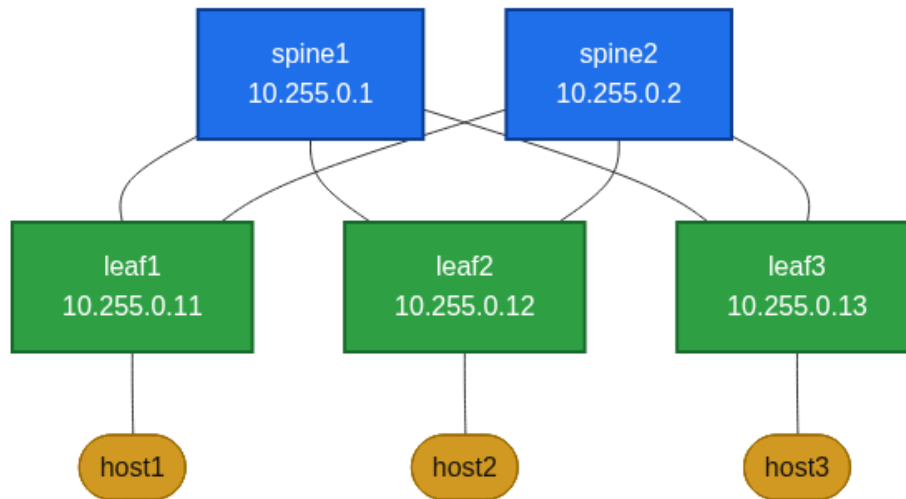


Figure 5: Topologie mixed

- **Pourquoi c'est mieux** : la session BGP repose sur la loopback, joignable par OSPF via **n'importe quel** chemin ☐ si un lien tombe, OSPF reroute et la session survit. Découplage robuste.
- **Vérification** : `show ip route 192.168.3.0/24` montre un next-hop loopback résolu en **2 chemins ECMP** (spine1 + spine2, les deux marqués *).

```

ECMP - show ip route (mixed leaf1)

$ docker exec clab-topo-mixed-leaf1 vtysh -c "show ip route 192.168.3.0/24"

Routing entry for 192.168.3.0/24
Known via "bgp", distance 200, metric 0, best
Last update 00:07:00 ago
Flags: Recursion iBGP Selected
Status: Installed
10.255.0.13 (recursive), weight 1
* 10.0.1.0, via eth1, weight 1
* 10.0.1.6, via eth2, weight 1
10.255.0.13 (duplicate nexthop removed) (recursive), weight 1
10.0.1.0, via eth1 (duplicate nexthop removed), weight 1
10.0.1.6, via eth2 (duplicate nexthop removed), weight 1

```

- Conclu-

sion : underlay simple (IGP) + overlay riche (BGP) = base de l'EVPN moderne.

7. EVPN/VXLAN (lab annexe, hors comparaison de débit)

EVPN/VXLAN a été monté séparément pour comprendre l'**overlay L2** : les hosts (10.10.10.x) sont sur un **même réseau L2 logique** alors qu'ils sont sur des leafs différents, grâce à l'encapsulation **VXLAN** (data-plane) pilotée par **BGP EVPN** (control-plane, distribue les MAC/IP). **VTEP** = la loopback de chaque leaf. > **Exclu du tableau comparatif** : overlay L2 + surcoût d'encapsulation (~50 o/paquet), pas comparable > au débit d'un routage L3 pur. (Inclus l'aurait rendu le tableau non iso — rigueur méthodologique.)

8. Bugs rencontrés & corrigés

Chaque bug a été **diagnostiqué puis corrigé** — c'est le cœur de l'apprentissage.

8.1 iBGP RR — next-hop non réécrit

Un Route Reflector ne modifie pas le next-hop par défaut ; next-hop-self seul ne s'applique pas aux routes réfléchies dans FRR. **Fix** : next-hop-self force sur chaque client du RR.

8.2 OSPF bloqué en 2-Way sur /31

OSPF en mode broadcast tente d'élire un DR/BDR inutile sur du point-à-point. **Fix** : ip ospf network point-to-point □ adjacence Full.

8.3 ECMP ne se répartit pas — hash kernel L3 □

2 chemins ECMP dans la FIB mais 100 % du trafic sur un seul spine, même avec 8 flux. **Cause** : hash kernel par défaut = **L3** (IP src+dst) □ même paire host □ host = même spine. **Fix** : fib_multipath_hash_policy=1 (hash **L4**, inclut les ports). **Mesuré** : 100/0 □ 62/38.

8.4 EVPN — TCP à 0 alors que le ping passe (MTU VXLAN) □

Ping OK, UDP OK, TCP = 0. **Cause** : vxlan100 en MTU 1500 vs hosts jumbo ; segments TCP pleins + encap VXLAN dépassent la MTU du tunnel □ **blackhole TCP** (PMTUD cassé). **Fix** : MTU vxlan100 à 9000 □ **TCP 0** □ **9.76 Gbit/s**. *Leçon* : « ça ping » ne prouve pas que ça marche, il faut **mesurer**.

9. Observabilité — Prometheus + Grafana + exporter maison

Stack (docker compose) : **Prometheus** (:9090, scrape 2 s) + **Grafana** (:3000) + node-exporter + **clab_exporter.py** (exporter Python écrit pour le projet).

Pourquoi un exporter maison : cAdvisor (l'outil standard) **échoue** ici (bug overlay2/cgroup v2, ne mappe pas les conteneurs containerlab □ aucune métrique réseau par nœud). Mon exporter entre dans la **netns** de chaque conteneur (nsenter), lit /proc/net/dev, et expose le débit RX/TX par conteneur/interface ; il publie aussi les résultats du benchmark.

Chaîne de la donnée : conteneurs → clab_exporter:9101 → Prometheus:9090 → Grafana:3000. **Dashboard unique** (4 sections) : comparaison (barres), trafic live par techno, **preuve ECMP** (RX par spine), ressources hôte.

10. Benchmark comparatif (la valeur ajoutée)

Technologie	TCP Gbit/s	UDP Gbit/s
iBGP-RR	11.12	8.96
mixed (OSPF+iBGP)	11.07	8.75
OSPF	11.06	8.97
eBGP	11.04	8.82

Interprétation : - **Tous à ~11 Gbit/s** (écart < 0,1 G = bruit de mesure) □ **le protocole de routage n'impacte pas le débit**. Une fois la route dans la FIB, le control-plane (BGP/OSPF) n'intervient plus ; le goulot est le **data-plane**



Figure 6: Grafana — comparaison du débit TCP par techno

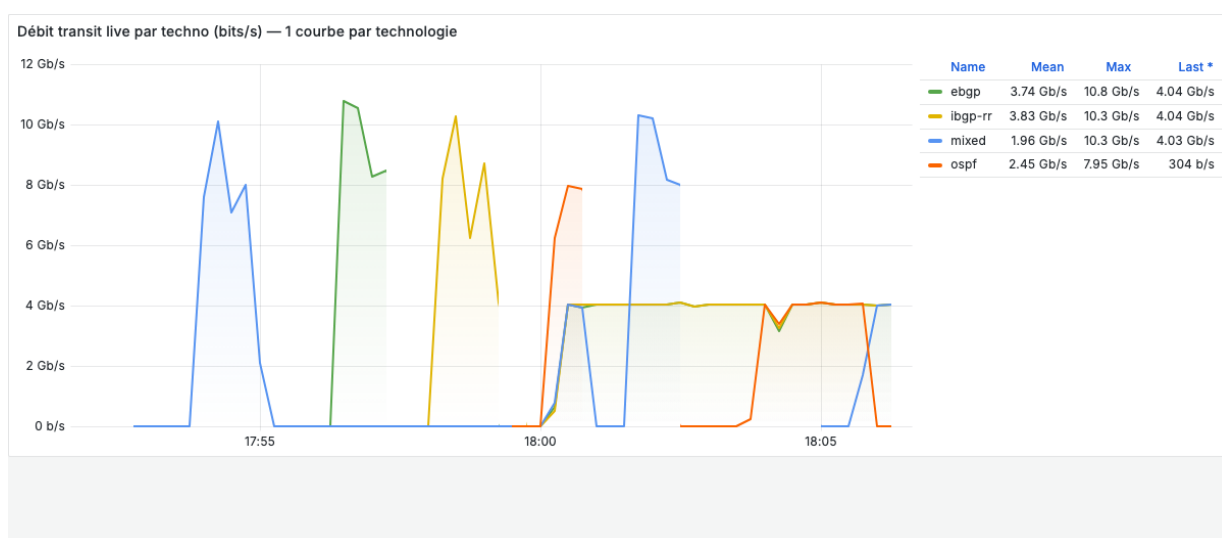


Figure 7: Grafana — trafic live, 1 courbe par technologie

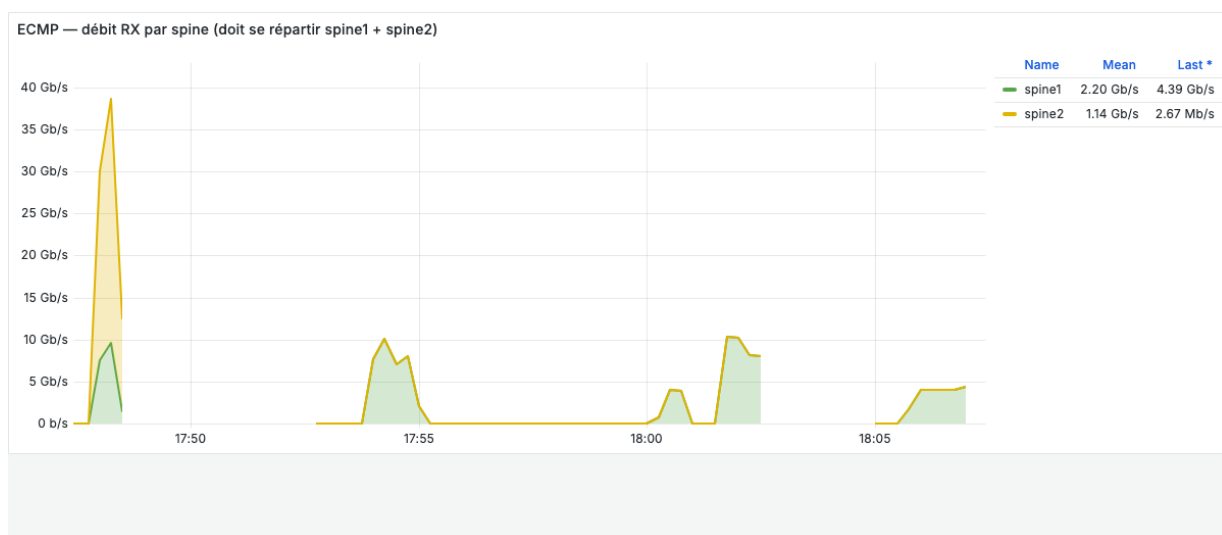


Figure 8: Grafana — preuve ECMP : RX réparti sur spine1 + spine2

(forwarding kernel/veth). - La vraie différence est **ailleurs** : convergence, scalabilité, policy — c'est ce qui guide le choix en production, pas le débit. - *Limite assumée* : 1 run par techno (indicatif) ; les écarts sont sous le seuil de bruit.

11. Mise en production & exposition

Isolation : le lab tourne sur **VM221 dédiée**, séparée de la **VM220 réservée à la production** (pas de contention CPU entre benchmarks et prod = bonne pratique).

NAT (DNAT) sur la passerelle pour exposer, depuis l'extérieur du réseau interne du lab : - le **dashboard Grafana** (:3000) pour la supervision ; - les **services web des conteneurs LXC**. Principe de **moindre exposition** : seul le port utile est traduit (IP_public:port → IP_interne:port), le reste du réseau interne reste masqué ; **masquerade** en sortie pour l'accès Internet des conteneurs. (*emplacement exact des règles : passerelle/VM dédiée — à intégrer depuis la config.*)

12. Conclusion & limites

Démonstré : un fabric leaf-spine en 4 technos validées de bout en bout ; **le protocole de routage n'impacte pas le débit** (chiffré) ; une **observabilité réelle** reliant mesure et visualisation ; **4 bugs réseau** diagnostiqués et corrigés ; une **mise en prod propre** (VM dédiée + NAT).

Limites : environnement virtuel (pas d'ASIC, débits non représentatifs d'un hardware réel) ; 1 run par techno ; fixes ECMP/MTU appliqués en runtime (à pérenniser en infra-as-code).

Perspectives : EVPN Type 5 (routage inter-VNI), BGP unnumbered, test de convergence chiffré (panne de spine).